

Venus Labs - Fixed Rate Vaults

Security Assessment

CertiK Assessed on May 12th, 2026





Certik Assessed on May 12th, 2026

Venus Labs - Fixed Rate Vaults

The security assessment was prepared by Certik.

Executive Summary

TYPES

Lending, Vault

ECOSYSTEM

Binance Smart Chain
(BSC)

METHODS

Manual Review, Static Analysis

LANGUAGE

Solidity

TIMELINE

Preliminary comments published on 05/01/2026

Final report published on 05/12/2026

Vulnerability Summary



15

Total Findings

14

Resolved

1

Multi-Sig & Timelock

0

Partially Resolved

0

Acknowledged

0

Declined

1 Centralization

1 Multi-Sig & Timelock

Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.

0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

0 Major

Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.

4 Medium

4 Resolved

Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

5 Minor

5 Resolved

Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

5 Informational

5 Resolved

Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | VENUS LABS - FIXED RATE VAULTS

Audit Summary

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

Findings

[VLF-01 : Centralization Risk](#)

[VLF-02 : `maxDeposit\(\)` And `maxMint\(\)` Can Advertise Unusable Residual Fundraising Capacity](#)

[VLF-03 : `depositCollateral\(\)` Does Not Support Cumulative Margin Deposits In `WaitingForMargin`](#)

[VLF-04 : One-Step State Catch-Up Can Allow Late `claimRaisedFunds\(\)` Execution](#)

[VLF-05 : Liquidation Parameters Are Not Validated Against A Solvency-Preserving Invariant](#)

[VLF-06 : Use `safeMint\(\)` Instead Of `mint\(\)`](#)

[VLF-07 : `closeVault\(\)` Relies On Documentation Instead Of Enforcing Completed Withdrawals](#)

[VLF-08 : Governance Transfer Approval Does Not Encode The Intended Recipient](#)

[VLF-09 : Vault Creation Does Not Verify Oracle Support For Configured Assets](#)

[VLF-10 : `deposit\(\)` And `mint\(\)` Silently Clamp Requests Instead Of Reverting When The Full Amount Cannot Be Accepted](#)

[VLF-11 : Unused Custom Error `ZeroAddress\(\)` In `InstitutionalLoanVault`](#)

[VLF-12 : Pause Mechanism Does Not Block `updateVaultState\(\)`-Driven Lifecycle Transitions](#)

[VLF-13 : Vault Accounting Assumes Non-Deflationary `supplyAsset` Transfers](#)

[VLF-14 : Protocol Share Settlement Handles PSR Update Failures Inconsistently](#)

[VLF-15 : Liquidation Does Not Reject Repayments That Round Down To Zero Seized Collateral](#)

Optimizations

[VLF-17 : Function Can Be Declared as `pure` or `view`](#)

[VLF-18 : `LiquidationAdapter` reconstructs vault liquidation data instead of using canonical vault outputs](#)

[VLF-19 : Redundant `unchecked` block reduces readability in a loop increment](#)

[VLF-20 : Canonical helper accessors are bypassed by direct storage reads in several paths](#)

Appendix

Disclaimer

CODEBASE | VENUS LABS - FIXED RATE VAULTS

Repository

<https://github.com/VenusProtocol/fixed-rate-vaults>

Commit

<https://github.com/VenusProtocol/fixed-rate-vaults/tree/dc57e93be55b1efdda035c3b97fae649ec3596a0/src>

<https://github.com/VenusProtocol/fixed-rate-vaults/tree/bd06150b9a73cf1a1f3287d558b60db3a8d37bf8/src/>

Audit Scope

The file in scope is listed in the appendix.

APPROACH & METHODS | VENUS LABS - FIXED RATE VAULTS

This audit was conducted for Venus Labs to evaluate the security and correctness of the smart contracts associated with the Venus Labs - Fixed Rate Vaults project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

FINDINGS | VENUS LABS - FIXED RATE VAULTS



15
Total Findings

0
Critical

1
Centralization

0
Major

4
Medium

5
Minor

5
Informational

This report has been prepared for Venus Labs to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 15 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
VLF-01	Centralization Risk	Centralization	Centralization	2/3 Multi-Sig, 24h Timelock
VLF-02	<code>maxDeposit()</code> And <code>maxMint()</code> Can Advertise Unusable Residual Fundraising Capacity	Logical Issue	Medium	Resolved
VLF-03	<code>depositCollateral()</code> Does Not Support Cumulative Margin Deposits In <code>waitingForMargin</code>	Logical Issue	Medium	Resolved
VLF-04	One-Step State Catch-Up Can Allow Late <code>claimRaisedFunds()</code> Execution	Logical Issue	Medium	Resolved
VLF-05	Liquidation Parameters Are Not Validated Against A Solvency-Preserving Invariant	Incorrect Calculation	Medium	Resolved
VLF-06	Use <code>_safeMint()</code> Instead Of <code>_mint()</code>	Logical Issue	Minor	Resolved
VLF-07	<code>closeVault()</code> Relies On Documentation Instead Of Enforcing Completed Withdrawals	Logical Issue	Minor	Resolved

ID	Title	Category	Severity	Status
VLF-08	Governance Transfer Approval Does Not Encode The Intended Recipient	Logical Issue	Minor	● Resolved
VLF-09	Vault Creation Does Not Verify Oracle Support For Configured Assets	Logical Issue	Minor	● Resolved
VLF-10	<code>deposit()</code> And <code>mint()</code> Silently Clamp Requests Instead Of Reverting When The Full Amount Cannot Be Accepted	Inconsistency	Minor	● Resolved
VLF-11	Unused Custom Error <code>ZeroAddress()</code> In <code>InstitutionalLoanVault</code>	Coding Issue	Informational	● Resolved
VLF-12	Pause Mechanism Does Not Block <code>updateVaultState()</code> - Driven Lifecycle Transitions	Logical Issue	Informational	● Resolved
VLF-13	Vault Accounting Assumes Non-Deflationary <code>supplyAsset</code> Transfers	Logical Issue	Informational	● Resolved
VLF-14	Protocol Share Settlement Handles PSR Update Failures Inconsistently	Inconsistency	Informational	● Resolved
VLF-15	Liquidation Does Not Reject Repayments That Round Down To Zero Seized Collateral	Incorrect Calculation	Informational	● Resolved

VLF-01 | Centralization Risk

Category	Severity	Location	Status
Centralization	● Centralization		● 2/3 Multi-Sig, 24h Timelock

Description

The protocol relies on a small set of privileged external governance roles to administer core components. In particular, the upgrade authority over upgradeable contracts such as `InstitutionalVaultController` and `LiquidationAdapter` can replace implementations and thereby change system behavior. In addition, ACM-gated governance functions in the controller and adapter can create vaults, pause or unpause vaults, close vaults, update key dependencies and risk parameters, and manage liquidation whitelists.

This is a trust and operational centralization risk rather than an implementation bug. If the privileged governance or upgrade accounts are compromised or misused, they could disrupt vault operation, alter liquidation behavior, or deploy upgraded logic with different semantics.

Recommendation

Document the privileged roles and secure them with strong operational controls such as multisig custody, timelocks for sensitive changes, and clear governance procedures for upgrades and parameter changes.

Alleviation

[Venus Labs, 05/06/2026]:

The ownership of all contracts will be transferred to the standard Venus Normal Timelock. ACM permissions will be assigned to specific timelocks based on the required level of control, normal, fast-track, or critical, and a guardian will be designated for critical setters. All permission assignments will be executed through a VIP and approved via Venus governance, accompanied by a community post to ensure transparency to users.

VLF-02 `maxDeposit()` And `maxMint()` Can Advertise Unusable Residual Fundraising Capacity

Category	Severity	Location	Status
Logical Issue	● Medium	BaseVault.sol (base): 499~500	● Resolved

Description

During `Fundraising`, `maxDeposit()` and `maxMint()` are derived from the remaining distance to `maxBorrowCap`, but `_deposit()` separately enforces `assets >= _config.minSupplierDeposit`. As a result, when the remaining fundraising capacity is positive but smaller than `minSupplierDeposit`, the vault can still report a positive maximum deposit or mint amount even though every such call will revert with `BelowMinimumDepositAmount()`.

This creates a boundary-condition liveness issue near the fundraising cap. The residual tranche becomes unusable, and callers receive a misleading signal that additional deposits are still possible. If fundraising later ends before `minBorrowCap` is reached, the vault may fail even though the interface continued to expose positive contribution capacity.

Recommendation

Align `maxDeposit()`, `maxMint()`, and `_deposit()` with the same minimum-deposit rule. If contributions below `minSupplierDeposit` are not allowed, return `0` whenever the remaining fundraising capacity is smaller than that minimum. Also consider reverting directly when a request exceeds the valid limit instead of silently clamping it to an amount that cannot succeed.

Alleviation

[Venus Labs, 05/06/2026]: The `_deposit()` minimum-deposit check is relaxed for the final tail only: when the remaining capacity is below `minSupplierDeposit`, a supplier can fill that residual amount in one shot. `maxDeposit()` already reports the residual, so it stays unchanged. and when a request exceeds the valid limit, clamping is the intended behaviour.

VLF-03 | `depositCollateral()` Does Not Support Cumulative Margin Deposits In `WaitingForMargin`

Category	Severity	Location	Status
Logical Issue	● Medium	institutional-vault/InstitutionalLoanVault.sol (base): 303~304, 329~330	● Resolved

Description

The documentation of `depositCollateral()` explicitly states that in `WaitingForMargin`, cumulative deposits must reach margin amount to transition to `MarginDeposited`. However, the implementation reverts with `InsufficientCollateral()` whenever the cumulative deposited amount after the current call is still below the required margin.

Because this revert happens in the same transaction, the collateral transfer and `_instRuntime.totalCollateralDeposited` update are rolled back. As a result, partial deposits do not persist, and the institution cannot build up the required margin across multiple transactions while the vault remains in `WaitingForMargin`.

Recommendation

Treat the margin threshold as a condition for the state transition only, not as a requirement for each individual `depositCollateral()` call in `WaitingForMargin`. Partial deposits should persist, and the vault should transition to `MarginDeposited` only once the accumulated amount reaches the required margin.

Alleviation

[Venus Labs, 05/12/2026]: The team heeded the advice and resolved the issue by updating the documentation in commit [a3502cb2333b8049e5638dd766c3fb9475f36584](#)

VLF-04 | One-Step State Catch-Up Can Allow Late `claimRaisedFunds()` Execution

Category	Severity	Location	Status
Logical Issue	● Medium	institutional-vault/InstitutionalLoanVault.sol (base): 387~391	● Resolved

Description

When a successful vault remains dormant in `Fundraising` past `openEndTime`, `_checkAndAdvanceState()` advances it only one step to `Lock` and returns immediately. It does not continue in the same call to catch up to the time-correct later state, even if `lockEndTime` or `settlementDeadline` has already passed.

Because `claimRaisedFunds()` calls `_checkAndAdvanceState()` and `_beforeClaimRaisedFunds()` only requires the vault to be in `Lock`, the institution can still claim `_runtime.totalRaised` on that first delayed interaction. This allows a stale intermediate `Lock` state to be used even after the claim window should already have been exceeded.

Recommendation

Make `_checkAndAdvanceState()` advance the vault to its time-correct state in a single call after prolonged inactivity. In particular, avoid returning immediately after `Fundraising -> Lock` when later lifecycle milestones have already passed. Additionally, enforce explicit timing or state conditions in `claimRaisedFunds()` so it cannot execute from a stale intermediate state.

VLF-05 | Liquidation Parameters Are Not Validated Against A Solvency-Preserving Invariant

Category	Severity	Location	Status
Incorrect Calculation	● Medium	institutional-vault/InstitutionalVaultController.sol (base): 546~554	● Resolved

Description

`liquidationThreshold`, `liquidationIncentive`, and `latePenaltyRate` are validated only by independent range checks. The protocol does not enforce a cross-parameter invariant ensuring that liquidation improves, or at least does not worsen, solvency under the same threshold model used to decide liquidatability.

This matters because liquidation eligibility is based on `liquidationThreshold`, while collateral seizure is based on either `liquidationIncentive` or `latePenaltyRate`. If `liquidationThreshold * liquidationIncentive >= 1e36`, or `liquidationThreshold * latePenaltyRate >= 1e36`, a repayment can remove enough collateral that the position's threshold-adjusted solvency does not improve and may become worse after liquidation.

Recommendation

Enforce cross-parameter validation during vault creation and parameter updates. In particular, reject configurations where `liquidationThreshold * liquidationIncentive >= 1e36` or `liquidationThreshold * latePenaltyRate >= 1e36`, and preferably require a safety margin below that boundary to account for rounding.

VLF-06 | Use `_safeMint()` Instead Of `_mint()`

Category	Severity	Location	Status
Logical Issue	● Minor	institutional-vault/InstitutionPositionToken.sol (base): 78~82	● Resolved

Description

According to [OpenZeppelin's ERC721 documentation](#), the use of `_mint()` is discouraged, use `_safeMint()` whenever possible.

Recommendation

Use `_safeMint()` whenever possible instead of `_mint()`

Alleviation

[Venus Labs, 05/06/2026]: Using `_mint()` is intentional. `_safeMint()` checks for `IERC721Receiver` and will revert if the recipient is a contract that doesn't implement it. Most institutional wallets (like Gnosis Safe) don't, so `_safeMint()` would fail in the common case.

VLF-07 `closeVault()` Relies On Documentation Instead Of Enforcing Completed Withdrawals

Category	Severity	Location	Status
Logical Issue	Minor	BaseVault.sol (base): 112~113	Resolved

Description

`closeVault()` can move a vault from `Matured`, `Failed`, or `Liquidated` to `Closed` without enforcing the condition stated in its documentation that governance should call it only after all suppliers have withdrawn their funds. As a result, the correctness of the close operation depends on off-chain discipline rather than an on-chain invariant.

This matters because `withdraw()` no longer accepts `Closed` as a valid state. If governance closes a vault prematurely, remaining supplier exits through the standard withdrawal flow will be blocked.

Recommendation

Enforce the documented precondition on-chain before allowing `closeVault()`. For example, require `totalSupply() == 0` before transitioning to `Closed`, or otherwise provide a dedicated post-close claim path for any remaining suppliers.

Alleviation

[Venus Labs, 05/06/2026]: `closeVault()` is ACM-gated through the governance flow. All preconditions (lenders fully redeemed, settlement complete, etc.) are verified off-chain before the call is made. A strict on-chain guard like `totalSupply() == 0` would actually hurt, dust shares left over from rounding (a few wei), or a single lender who never bothers to redeem, would permanently block governance from closing the vault.

VLF-08 Governance Transfer Approval Does Not Encode The Intended Recipient

Category	Severity	Location	Status
Logical Issue	● Minor	institutional-vault/InstitutionPositionToken.sol (base): 97~98	● Resolved

Description

`approveTransfer()` only marks `transferApproved[tokenId] = true` and does not bind the approval to a specific recipient. Once this flag is set, any otherwise authorized ERC-721 transfer can consume the approval, regardless of who receives the token.

If governance intends transfer approval to reflect acceptance of the next holder, this design does not enforce that intent on-chain. The approved transfer window authorizes a token movement, but not a specific handoff.

Recommendation

Bind transfer approval to the intended recipient, and reject transfers that do not match the governance-approved destination. If governance is also expected to approve the exact caller, encode that constraint as well.

VLF-09 | Vault Creation Does Not Verify Oracle Support For Configured Assets

Category	Severity	Location	Status
Logical Issue	Minor	institutional-vault/InstitutionalVaultController.sol (base): 531~533	Resolved

Description

`createVault()` validates basic configuration ranges and addresses, but it does not verify that the configured oracle supports pricing for either `instConfig.collateralAsset` or `vaultConfig.supplyAsset`. The vault later depends on oracle prices for both assets in lifecycle and liquidation logic. For example, `_enterLock()` calls `_getCollateralValueUSD()` and reverts with `InvalidOraclePrice()` if the collateral price is `0`, while `_calculateSeizeAmount()` depends on nonzero prices for both the supply and collateral assets.

As a result, a vault can be created successfully in a configuration that later cannot progress or operate correctly once price-dependent logic is reached.

Recommendation

Validate oracle support for all configured assets during vault creation, including both `collateralAsset` and `supplyAsset`. Reject vault configurations whose required asset prices are unavailable or return `0`.

VLF-10 | `deposit()` And `mint()` Silently Clamp Requests Instead Of Reverting When The Full Amount Cannot Be Accepted

Category	Severity	Location	Status
Inconsistency	Minor	BaseVault.sol (base): 222~223, 244~245	Resolved

Description

`deposit()` and `mint()` reduce user-supplied inputs to `maxDeposit()` and `maxMint()` respectively instead of reverting when the requested amount exceeds what the vault can currently accept. This behavior is inconsistent with [ERC-4626 expectations](#) that `deposit()` and `mint()` should revert if the full requested amount cannot be executed.

As a result, integrators that rely on standard ERC-4626 all-or-nothing semantics may observe partial execution behavior that differs from the specification.

Recommendation

Make `deposit()` and `mint()` revert when the requested `assets` or `shares` exceed the currently acceptable amount, instead of silently clamping the request to a smaller value.

Alleviation

[Venus Labs, 05/06/2026]: The clamp is intentional. We don't want a supplier's tx to fail at the very end of fundraising because they happened to overshoot the remaining cap by a small amount, the vault accepts what it can and the caller reads back the actual assets / shares returned.

VLF-11 | Unused Custom Error `ZeroAddress()` In `InstitutionalLoanVault`

Category	Severity	Location	Status
Coding Issue	● Informational	institutional-vault/InstitutionalLoanVault.sol (base): 74	● Resolved

Description

The contract declares the custom error `ZeroAddress()` but never uses it in any revert path. This leaves dead code in the ABI surface and can mislead integrators or reviewers into assuming there is zero-address validation in `InstitutionalLoanVault()` when there is none.

Recommendation

Remove `ZeroAddress()` if it is unnecessary, or add the intended zero-address validation and revert with `ZeroAddress()` at the appropriate call site.

VLF-12 | Pause Mechanism Does Not Block `updateVaultState()` - Driven Lifecycle Transitions

Category	Severity	Location	Status
Logical Issue	● Informational	BaseVault.sol (base): 176~177	● Resolved

Description

`updateVaultState()` remains callable while the vault is paused because it does not use `whenNotPaused()` or `whenNotCompletelyPaused()`. As a result, third parties can still trigger deterministic lifecycle transitions while `partialPause()` or `completePause()` is active.

This does not let an attacker choose arbitrary state transitions, since `_checkAndAdvanceState()` still follows the contract's predefined rules. However, it creates an inconsistency in the pause model: several state-changing operations are blocked during pause, while lifecycle advancement remains externally callable.

Recommendation

Clarify whether pause is intended to stop only user operations or to freeze lifecycle progression entirely. If full freeze semantics are intended, gate `updateVaultState()` with the appropriate pause modifier. Otherwise, document this behavior explicitly so the pause model is unambiguous.

Alleviation

[Venus Labs, 05/06/2026]: `updateVaultState()` only runs deterministic, time-based transitions, there's nothing a caller can steer. Gating it with `whenNotCompletelyPaused` would freeze the on-chain state from advancing while the wall clock keeps moving, which makes the vault's reported state diverge from its time-correct state. That's bad for external viewers (explorers, frontends, integrators) and gives no extra safety benefit every user-facing action that could touch funds (deposits, withdrawals, repays, claims, liquidations) is already gated by the pause modifiers, so the state machine itself is safe to keep advancing.

VLF-13 | Vault Accounting Assumes Non-Deflationary `supplyAsset` Transfers

Category	Severity	Location	Status
Logical Issue	● Informational	BaseVault.sol (base): 501~502, 560~562	● Resolved

Description

The vault's `supplyAsset` accounting uses nominal transfer amounts rather than observed balance deltas. In particular, `_deposit()` increases `_runtime.totalRaised` using the requested `assets` amount, and repayment logic reduces debt by the nominal repaid amount. If a deflationary or fee-on-transfer token is configured as `supplyAsset`, the contract may record more assets as received than were actually transferred.

This can distort fundraising, debt, and settlement accounting, since the protocol implicitly assumes standard non-deflationary ERC-20 behavior but does not enforce that assumption.

Collateral accounting is also affected.

Recommendation

Restrict `supplyAsset` to standard non-deflationary tokens, or update deposit and repayment flows to account using actual balance deltas instead of requested transfer amounts.

Alleviation

[Venus Labs, 05/06/2026]: We don't intend to support deflationary or fee-on-transfer tokens. The vault is meant to be deployed only with standard ERC-20 supply and collateral assets, and asset onboarding is governance-gated non-deflationary behavior is part of the asset-eligibility check during VIP review. Adding balance-delta accounting would only support a class of tokens the protocol explicitly doesn't onboard.

VLF-14 | Protocol Share Settlement Handles PSR Update Failures Inconsistently

Category	Severity	Location	Status
Inconsistency	● Informational	institutional-vault/LiquidationAdapter.sol (base): 211~215	● Resolved

Description

`BaseVault._settleProtocolShare()` and `LiquidationAdapter.sweepProtocolShareToReserve()` interact with `IProtocolShareReserve.updateAssetsState()` using different failure-handling strategies. In `BaseVault`, the PSR accounting update is wrapped in `try/catch`, allowing the function to continue even if the notification fails. In `LiquidationAdapter`, the same external PSR update is performed without `try/catch`, so any revert aborts the entire sweep.

This inconsistency can make protocol-share accounting behavior harder to reason about operationally. Similar PSR update failures may be tolerated in one flow but fully revert another, even though both paths are performing reserve-accounting notifications after transferring assets to PSR.

Recommendation

Use a consistent error-handling policy for PSR accounting updates across settlement and sweep flows. If failures should be tolerated, both paths should expose a recovery or retry mechanism. If failures should be strict, both paths should revert atomically.

Alleviation

[Venus Labs, 05/06/2026]: The asymmetry is intentional. In the vault, a PSR notification failure must not block lender redemptions the share is still recoverable via `sweepProtocolShareToReserve()` and `PSRNotificationFailed` keeps the failure observable. In `sweepProtocolShareToReserve()` itself, the function's only job is the PSR round-trip, so a revert is the correct signal that it didn't complete and lets governance retry.

VLF-15 | Liquidation Does Not Reject Repayments That Round Down To Zero Seized Collateral

Category	Severity	Location	Status
Incorrect Calculation	● Informational	institutional-vault/InstitutionalLoanVault.sol (base): 744~745	● Resolved

Description

`_calculateSeizeAmount()` uses integer division and can round the seized collateral amount down to `0` for sufficiently small repayments. The liquidation flow does not reject this case. As a result, a liquidator can make a positive repayment, reduce the vault's debt, and still receive no collateral because the computed seize amount is zero.

This creates a dust-liquidation edge case where the transaction can succeed with economically unfavorable results for the liquidator.

Recommendation

Reject liquidations when a positive repayment would result in `0` seized collateral. Enforcing a minimum repay amount or reverting when `seizeAmount == 0` would prevent zero-output liquidations.

Alleviation

[Venus Labs, 05/06/2026]: In practice a liquidator won't submit a repayment small enough that the seize rounds to zero there's no reason to spend gas on a tx that returns no collateral. Adding an on-chain `seizeTokens > 0` guard would only protect against a self-harming action that nobody has an incentive to take.

OPTIMIZATIONS | VENUS LABS - FIXED RATE VAULTS

ID	Title	Category	Severity	Status
VLF-17	Function Can Be Declared As <code>pure</code> Or view	Gas Optimization, Coding Style	Optimization	● Resolved
VLF-18	<code>LiquidationAdapter</code> Reconstructs Vault Liquidation Data Instead Of Using Canonical Vault Outputs	Code Optimization	Optimization	● Acknowledged
VLF-19	Redundant <code>unchecked</code> Block Reduces Readability In A Loop Increment	Code Optimization	Optimization	● Resolved
VLF-20	Canonical Helper Accessors Are Bypassed By Direct Storage Reads In Several Paths	Code Optimization	Optimization	● Resolved

VLF-17 | Function Can Be Declared As `pure` Or `view`

Category	Severity	Location	Status
Gas Optimization, Coding Style	● Optimization	InstitutionPositionToken.sol: 63; InstitutionalVaultContro ller.sol: 561; LiquidationAdapter.sol: 318	● Resolved

Description

`renounceOwnership()` functions do not require modifying the contract state, nor do they need to receive Ether. However, these functions are not labeled with `pure` or `view` modifiers.

Recommendation

To enhance code readability and safety, as well as enable the compiler to conduct stricter checks during compilation, we recommend utilizing either `pure` or `view` on these functions.

VLF-18 | `LiquidationAdapter` Reconstructs Vault Liquidation Data Instead Of Using Canonical Vault Outputs

Category	Severity	Location	Status
Code Optimization	● Optimization	institutional-vault/LiquidationAdapter.sol (base): 280~282	● Acknowledged

Description

`LiquidationAdapter` reconstructs liquidation-related data by reading `config()`, `institutionalConfig()`, and `riskConfig()` from the vault and then deriving values such as the collateral split from `totalSeized` and the current incentive multiplier. This breaks encapsulation and duplicates logic that conceptually belongs to the vault-side liquidation flow.

As a result, the adapter is tightly coupled to vault internals and must remain synchronized with vault liquidation math and asset layout. This makes the design more brittle and increases maintenance overhead if liquidation accounting changes in the future.

Recommendation

Expose narrower canonical outputs from the vault instead of reconstructing liquidation state in the adapter. For example, provide dedicated getters such as `collateral()` for the collateral asset, and have `liquidate()` / `liquidateOverdueVault()` return the exact values the adapter needs, such as the realized incentive-related amounts, rather than requiring the adapter to recompute them externally.

Alleviation

[Venus Labs, 05/06/2026]: Split is intentional the vault owns lifecycle and liquidation math, the adapter handles PSR routing.

VLF-19 | Redundant `unchecked` Block Reduces Readability In A Loop Increment

Category	Severity	Location	Status
Code Optimization	 Optimization	institutional-vault/InstitutionalVaultController.sol (base): 491 ~494	 Resolved

Description

Due to the improvements introduced in Solidity version 0.8.22, the use of an explicit `unchecked` block for incrementing loop counters is now generally redundant, as the compiler automatically applies the relevant optimizations where applicable.

In this case, keeping the loop in its ordinary form would improve readability without materially changing the generated behavior.

Recommendation

Remove the explicit `unchecked` block and rely on the compiler's loop increment optimization where applicable.

VLF-20 Canonical Helper Accessors Are Bypassed By Direct Storage Reads In Several Paths

Category	Severity	Location	Status
Code Optimization	● Optimization	BaseVault.sol (base): 557; institutional-vault/InstitutionalLoanVault.sol (base): 689, 733	● Resolved

Description

The codebase already defines canonical helper accessors such as `asset()` and `_outstandingDebt()`, but some paths still bypass them and read the underlying storage directly. For example, `_receiveRepayment()` reads `_runtime.totalDebt` directly, while other debt-related logic consistently uses `_outstandingDebt()`. Similarly, `_getDebtValueUSD()` and `_calculateSeizeAmount()` access `address(_config.supplyAsset)` directly instead of using `asset()`.

This does not currently change behavior, but it weakens consistency and makes future refactors more fragile. If helper functions are later extended with additional logic, validation, or normalization, direct storage reads can silently diverge from the intended source of truth.

Recommendation

Use canonical helper accessors consistently throughout the codebase. In particular, prefer `_outstandingDebt()` over direct reads of `_runtime.totalDebt`, and prefer `asset()` over direct access to `_config.supplyAsset`. Similar cleanup can also be applied in places where the same value is exposed through a dedicated helper but re-read from storage instead.

APPENDIX | VENUS LABS - FIXED RATE VAULTS

Audit Scope

CertiKProject/certik-audit-projects

- institutional-vault/LiquidationAdapter.sol
- institutional-vault/InstitutionPositionToken.sol
- institutional-vault/InstitutionalLoanVault.sol
- institutional-vault/InstitutionalVaultController.sol
- BaseVault.sol
- interfaces/IInstitutionPositionToken.sol
- interfaces/IInstitutionalLoanVault.sol
- interfaces/IInstitutionalVaultController.sol
- interfaces/IInstitutionalVaultTypes.sol
- interfaces/ILiquidationAdapter.sol
- interfaces/IProtocolShareReserve.sol
- interfaces/IResilientOracle.sol
- interfaces/IVaultController.sol
- interfaces/IVaultTypes.sol

Finding Categories

Categories	Description
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.

Categories	Description
Incorrect Calculation	Incorrect Calculation findings are about issues in numeric computation such as rounding errors, overflows, out-of-bounds and any computation that is not intended.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Gas Optimization	Gas Optimization findings do not affect the functionality of the code but generate different, more optimal EVM opcodes resulting in a reduction on the total gas cost of a transaction.
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Code Optimization	No description available.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.

Venus Labs - Fixed Rate Vaults Security Assessment | CertiK Assessed on May 12th, 2026 | Copyright © CertiK

